

AprilTag-based Trajectory Tracking

for the LIMOS Robot

Joao Pedro Martins do Lago Reis and Yann Kelvem da Silva Ramos

Master's degree in artificial perception and robotics

December 9, 2025

Outline

Problem

Challenges

Project goal

Phases

Problem description

Problem to solve

- ▶ We need a robot that can **learn a path by being guided**.

Problem to solve

- ▶ We need a robot that can **learn a path by being guided**.
- ▶ Later it must **drive that same path alone**.

Problem to solve

- ▶ We need a robot that can **learn a path by being guided**.
- ▶ Later it must **drive that same path alone**.
- ▶ It has to work in normal indoor rooms without fuss.

Why this is challenging

- ▶ **Hardcoded paths:** someone types coordinates by hand.

Why this is challenging

- ▶ **Hardcoded paths**: someone types coordinates by hand.
- ▶ **No flexibility**: any small change means rebuilding code.

Why this is challenging

- ▶ **Hardcoded paths:** someone types coordinates by hand.
- ▶ **No flexibility:** any small change means rebuilding code.
- ▶ **High barrier:** only experts can tweak routes.

Chosen setup

Selected Platform

- ▶ Robot: **LIMOS** with wheels and camera.



Fig. 1 — LIMOS mobile robot

Selected Platform

- ▶ Robot: **LIMOS** with wheels and camera.
- ▶ Tags: **AprilTag** to tell where the robot is.

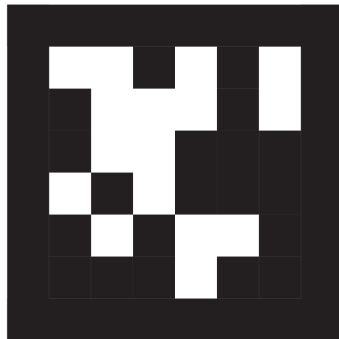


Fig. 2 — AprilTag visual marker

Selected Platform

- ▶ Robot: **LIMOS** with wheels and camera.
- ▶ Tags: **AprilTag** to tell where the robot is.
- ▶ Together they keep the pose steady while we teach and repeat.



Fig. 1 — LIMOS robot



Fig. 2 — AprilTag marker

Project Goal

Our Goal

- ▶ Build a simple **teach and repeat** loop.

Project Goal

Our Goal

- Build a simple **teach and repeat** loop.

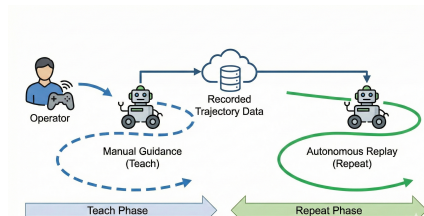


Fig.3 — High-level teach and repeat concept

Project Goal

Our Goal

- ▶ Build a simple **teach and repeat** loop.
- ▶ **No coding**: you just guide the robot by hand.

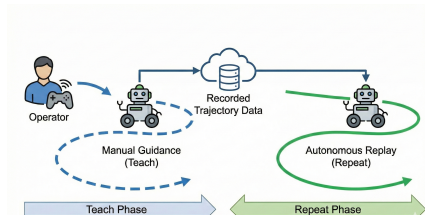


Fig.3 — High-level teach and repeat concept

Project Goal

Our Goal

- ▶ Build a simple **teach and repeat** loop.
- ▶ **No coding**: you just guide the robot by hand.
- ▶ **Auto replay**: it drives the same path accurately later.

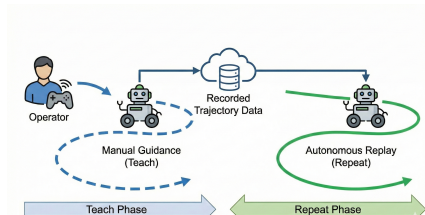


Fig.3 — High-level teach and repeat concept

Phase 1: TF setup and perception

Goal: Prepare clean TF data for simulation

► **Step 1 — Put tags on the map.**

We save where each AprilTag sits:

$$T_{\text{world}}^{\text{tag}}$$

This stays fixed for every next step.

Phase 1: TF setup and perception

Goal: Prepare clean TF data for simulation

► **Step 1 — Put tags on the map.**

We save where each AprilTag sits:

$$T_{\text{world}}^{\text{tag}}$$

This stays fixed for every next step.

► **Step 2 — Drive the robot once.**

The simulator gives the robot pose while it follows a smooth curve:

$$T_{\text{world}}^{\text{camera_link}}(t)$$

Phase 1: TF setup and perception

Goal: Prepare clean TF data for simulation

► **Step 1 — Put tags on the map.**

We save where each AprilTag sits:

$$T_{\text{world}}^{\text{tag}}$$

This stays fixed for every next step.

► **Step 2 — Drive the robot once.**

The simulator gives the robot pose while it follows a smooth curve:

$$T_{\text{world}}^{\text{camera_link}}(t)$$

► **Step 3 — Use the real camera frame.**

We switch to the optical frame (how the camera really looks):

$$T_{\text{world}}^{\text{camera_optical}}(t) = T_{\text{world}}^{\text{camera_link}}(t) \cdot T_{\text{camera_link}}^{\text{camera_optical}}$$

Phase 1: TF setup and perception

Goal: Prepare clean TF data for simulation

► **Step 1 — Put tags on the map.**

We save where each AprilTag sits:

$$T_{\text{world}}^{\text{tag}}$$

This stays fixed for every next step.

► **Step 2 — Drive the robot once.**

The simulator gives the robot pose while it follows a smooth curve:

$$T_{\text{world}}^{\text{camera_link}}(t)$$

► **Step 3 — Use the real camera frame.**

We switch to the optical frame (how the camera really looks):

$$T_{\text{world}}^{\text{camera_optical}}(t) = T_{\text{world}}^{\text{camera_link}}(t) \cdot T_{\text{camera_link}}^{\text{camera_optical}}$$

► **Step 4 — Clean tag detections.**

From the map and pose we get, for each tag:

$$T_{\text{camera_optical}}^{\text{tag}}(t)$$



Phase 2: Simulation and dataset generation

Goal: Create a realistic noisy dataset for testing

► **Step 1 — Simulate a clean path.**

We make a smooth curve and log the true pose:

$$T_{\text{world}}^{\text{camera_optical}}(t)$$

Phase 2: Simulation and dataset generation

Goal: Create a realistic noisy dataset for testing

► **Step 1 — Simulate a clean path.**

We make a smooth curve and log the true pose:

$$T_{\text{world}}^{\text{camera_optical}}(t)$$

► **Step 2 — Compute perfect tag reads.**

For every tag and time we compute the clean detection:

$$T_{\text{camera_optical}}^{\text{tag}}(t)$$

Phase 2: Simulation and dataset generation

Goal: Create a realistic noisy dataset for testing

► **Step 1 — Simulate a clean path.**

We make a smooth curve and log the true pose:

$$T_{\text{world}}^{\text{camera_optical}}(t)$$

► **Step 2 — Compute perfect tag reads.**

For every tag and time we compute the clean detection:

$$T_{\text{camera_optical}}^{\text{tag}}(t)$$

► **Step 3 — Add noise like real life.**

We add Gaussian noise so readings look imperfect, like real sensors.

Phase 2: Simulation and dataset generation

Goal: Create a realistic noisy dataset for testing

► **Step 1 — Simulate a clean path.**

We make a smooth curve and log the true pose:

$$T_{\text{world}}^{\text{camera_optical}}(t)$$

► **Step 2 — Compute perfect tag reads.**

For every tag and time we compute the clean detection:

$$T_{\text{camera_optical}}^{\text{tag}}(t)$$

► **Step 3 — Add noise like real life.**

We add Gaussian noise so readings look imperfect, like real sensors.

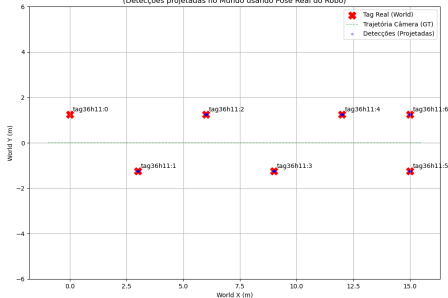
► **Step 4 — Save the dataset.**

For each time we save:

- the true pose $T_{\text{world}}^{\text{camera_optical}}(t)$
- the noisy tag detections $T_{\text{camera_optical}}^{\text{tag}}(t)$

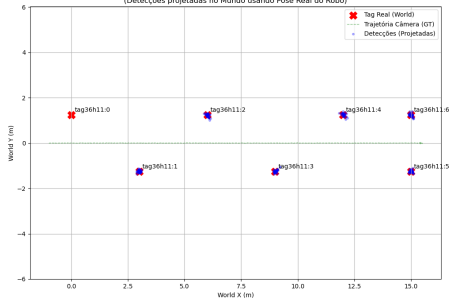
Phase 2: Visual — Straight trajectory with tags

Realização do Dataset: /home/jpdark/limo_autonomy_project_M2_PAR/limo_ws/validation_archive/test/reto/datasets/trajectory_reto_6_tags_datas
(Detecções projetadas no Mundo usando Pose Real do Robô)



Clean straight-path dataset (6 tags)

Realização do Dataset: /home/jpdark/limo_autonomy_project_M2_PAR/limo_ws/validation_archive/test/reto/datasets/trajectory_reto_6_tags_noisy_da
(Detecções projetadas no Mundo usando Pose Real do Robô)

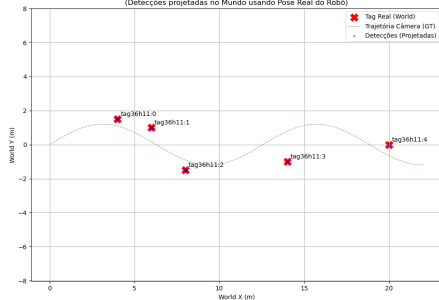


Noisy straight-path dataset

Fig. 5.1 — Straight trajectory: clean (left) and noisy (right)

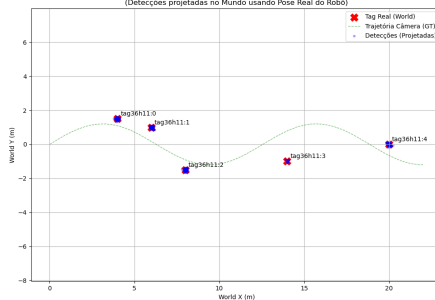
Phase 2: Visual — S-curve trajectory with 5 tags

Visualização do Dataset: /home/jpdark/limo_autonomy_project_M2_PAR/limo_ws/validation_archive/test/s_curve/datasets/trajectory_s_5_tags_datos
(Detecções projetadas no Mundo usando Pose Real do Robô)



Clean S-curve dataset (5 tags)

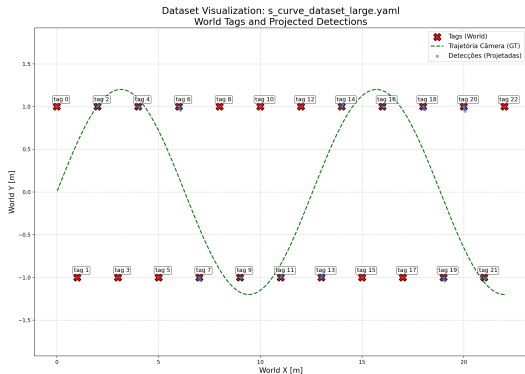
Visualização do Dataset: /home/jpdark/limo_autonomy_project_M2_PAR/limo_ws/validation_archive/test/s_curve/datasets/trajectory_s_5_tags_noisy_datos
(Detecções projetadas no Mundo usando Pose Real do Robô)



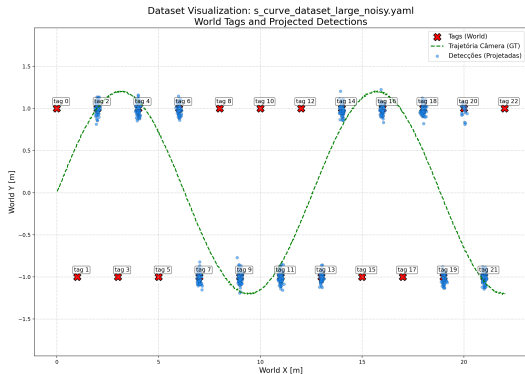
Noisy S-curve dataset (5 tags)

Fig. 5.2 — S-curve with 5 tags: clean (left) and noisy (right)

Phase 2: Visual — Dense S-curve trajectory



Clean S-curve dataset (dense tags)



Noisy S-curve dataset (dense tags)

Fig. 5.3 — S-curve with many tags: clean (left) and noisy (right)

Phase 3: Trajectory Optimization & Recovery

Goal: Fix drift with a simple optimizer

- ▶ **Path recovery:** Align the noisy run with the taught path.

Phase 3: Trajectory Optimization & Recovery

Goal: Fix drift with a simple optimizer

- ▶ **Path recovery:** Align the noisy run with the taught path.
- ▶ **Pose graph:** Reduce error between waypoints and simple constraints.

Phase 3: Trajectory Optimization & Recovery

Goal: Fix drift with a simple optimizer

- ▶ **Path recovery:** Align the noisy run with the taught path.
- ▶ **Pose graph:** Reduce error between waypoints and simple constraints.
- ▶ **Drift fix:** Auto-correct rotation and position drift.

Recovery the Optimal Path Sinusoidal Trajectory

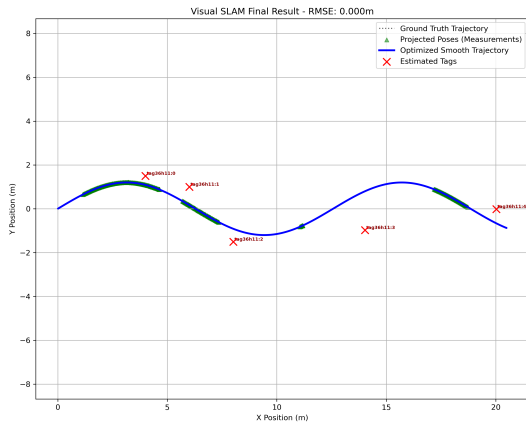


Fig. 7.2 — Trajectory Recovery via Optimization

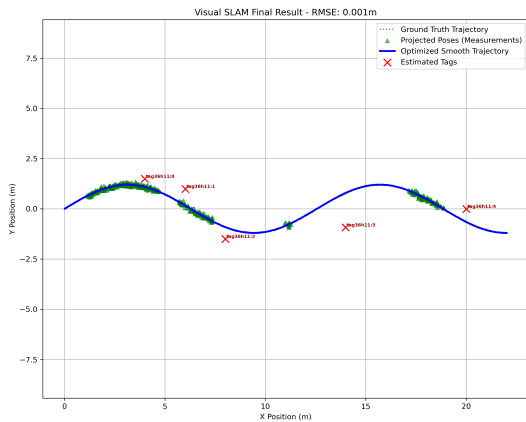


Fig. 7.1 — Simulated Dataset with Gaussian Noise

Quantitative Analysis Sinusoidal Trajectory

Case 1: Optimization without NOISE (Ideal)

Metric	Value
Trajectory RMSE	0.0000 m
Avg. Tag Position Error	0.0164 m

Tag Estimation Error (Ideal)

Tag Name	X Error	Y Error	Z Error
tag36h11:0	0.00	0.00	0.00
tag36h11:1	0.00	0.00	0.00
tag36h11:2	0.01	-0.01	0.00
tag36h11:3	0.03	0.00	0.00
tag36h11:4	0.02	-0.02	0.00

Case 2: Optimization with NOISE (6 cm)

Metric	Value
Trajectory RMSE	0.0007 m
Avg. Tag Position Error	0.0239 m

Tag Estimation Error

Tag Name	X Error	Y Error	Z Error
tag36h11:0	0.00	0.01	0.01
tag36h11:1	0.01	0.00	0.00
tag36h11:2	0.00	-0.04	0.01
tag36h11:3	-0.01	0.07	0.00
tag36h11:4	0.00	-0.01	0.00

Tables — Comparison of Optimization Results with and without noise.

Recovery the Optimal Path Sinusoidal Trajectory

The scenario with 23 tags

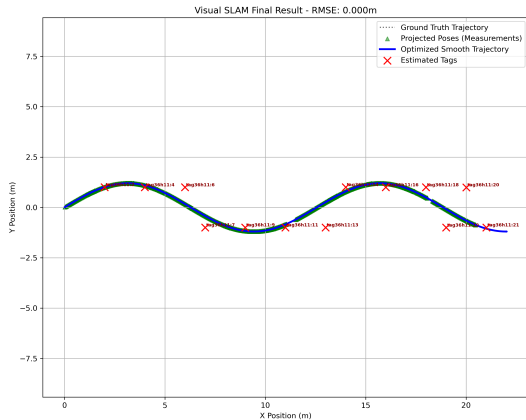


Fig. 8.1 — Trajectory Recovery via Optimization

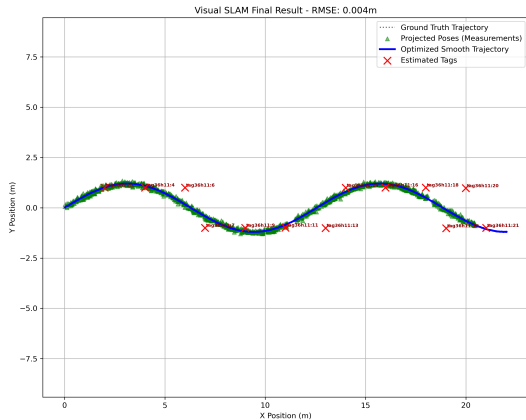


Fig. 8.2 — Simulated Dataset with Gaussian Noise

Quantitative Analysis Sinusoidal Trajectory

The scenario with 23 tags

Case 1: Optimization without NOISE

Metric	Value
Trajectory RMSE	0.0002 m
Avg. Tag Position Error	0.0019 m

Tag Estimation Error (Sample)

Tag Name	X Error	Y Error	Z Error
tag36h11:2	0.00	0.00	0.00
tag36h11:7	0.00	0.00	0.00
tag36h11:13	0.00	0.00	0.00
tag36h11:18	0.00	0.00	0.00
tag36h11:21	0.00	0.00	0.00

Case 2: Optimization with NOISE (6 cm)

Metric	Value
Trajectory RMSE	0.0037 m
Avg. Tag Position Error	0.0134 m

Tag Estimation Error (Sample)

Tag Name	X Error	Y Error	Z Error
tag36h11:2	0.01	0.01	0.00
tag36h11:7	0.00	0.00	0.00
tag36h11:13	0.01	0.01	0.00
tag36h11:18	0.00	0.00	0.01
tag36h11:21	0.01	0.00	0.02

Tables — Comparison of Optimization Results with and without noise (Sample of 5 Tags).

Recovery the Optimal Path: Straight Trajectory

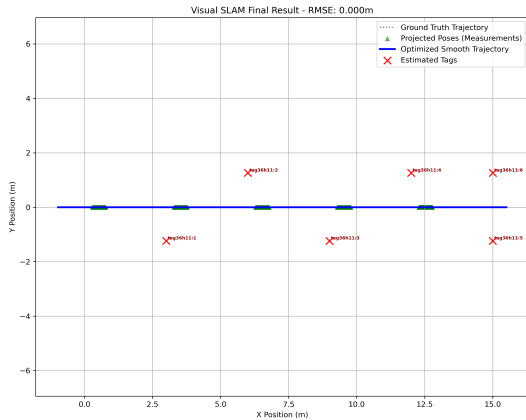


Fig. 8.1 — Trajectory Recovery via Optimization

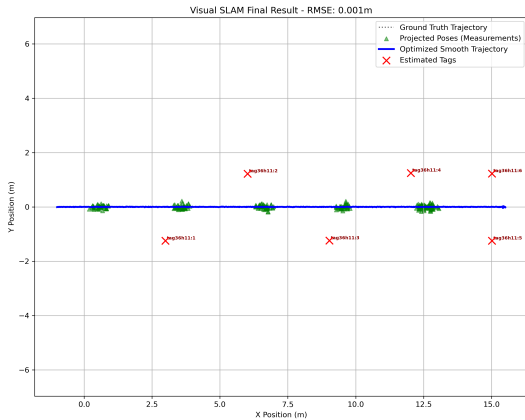


Fig. 8.2 — Simulated Dataset with Gaussian Noise

Quantitative Analysis: Straight Trajectory

Case 1: Optimization without NOISE

Metric	Value
Trajectory RMSE	0.0017 m
Avg. Tag Position Error	0.0000 m

Tag Estimation Error (Sample)

Tag Name	X Error	Y Error	Z Error
tag36h11:0	0.00	0.00	0.00
tag36h11:1	0.00	0.00	0.00
tag36h11:2	0.00	0.00	0.00
tag36h11:3	0.00	0.00	0.00
tag36h11:4	0.00	0.00	0.00

Case 2: Optimization with NOISE (6 cm)

Metric	Value
Trajectory RMSE	0.017 m
Avg. Tag Position Error	0.01912 m

Tag Estimation Error (Sample)

Tag Name	X Error	Y Error	Z Error
tag36h11:0	0.01	0.01	0.00
tag36h11:1	0.00	0.00	0.00
tag36h11:2	0.01	0.01	0.00
tag36h11:3	0.00	0.00	0.01
tag36h11:4	0.01	0.00	0.02

Tables — Comparison of Optimization Results with and without noise (Sample of 5 Tags).

Rubber duck check — Straight path

Pato de borracha: comparar sem ruído vs. com ruído; se o pato não enxerga a mesma forma, algo está errado.

```
=====
                        RESULTADOS FINAIS
=====
RMSE da Trajetória (Alinhada pelas Tags): 0.0017 m
Salvando plot 'slam_final_graph.png'...
Plot salvo.

Salvando 'dataset_otimizado_final.yaml'...
Arquivo YAML salvo com sucesso!

=====
                        TAG ESTIMATION ERROR ANALYSIS
=====
Tag Name      | Est (x,y,z)      | True (x,y,z)      | Error (m)
-----
tag36h11:1    | 3.00,-1.25,0.50   | 3.00,-1.25,0.50   | 0.0000
tag36h11:2    | 6.00,1.25,0.50    | 6.00,1.25,0.50    | 0.0000
tag36h11:3    | 9.00,-1.25,0.50   | 9.00,-1.25,0.50   | 0.0000
tag36h11:4    | 12.00,1.25,0.50   | 12.00,1.25,0.50   | 0.0000
tag36h11:5    | 15.00,-1.25,0.50  | 15.00,-1.25,0.50  | 0.0000
tag36h11:6    | 15.00,1.25,0.50   | 15.00,1.25,0.50   | 0.0000
-----
Average Tag Position Error: 0.0000 m
=====
```

Clean dataset — straight trajectory

```
=====
Generating initial estimate...

[STEP 1/2] Optimizing Map (Tags)...

[STEP 2/2] Optimizing Smooth Trajectory...
STEP 2 DONE | Time: 0.37s | Cost: 1559.7524

=====
                        FINAL RESULTS
=====
Trajectory RMSE: 0.0007 m
Saving plot to 'slam_final_graph.png'...
Plot saved.

Saving 'dataset_otimizado.yaml' with dataset structure...
File 'dataset_otimizado.yaml' saved successfully (Robot-Ready)!

=====
                        TAG ESTIMATION ERROR ANALYSIS
=====
Tag Name      | Est (x,y,z)      | True (x,y,z)      | Error (m)
-----
tag36h11:1    | 2.99,-1.25,0.51   | 3.00,-1.25,0.50   | 0.0115
tag36h11:2    | 6.01,1.22,0.49    | 6.00,1.25,0.50    | 0.0337
tag36h11:3    | 9.03,-1.24,0.48    | 9.00,-1.25,0.50    | 0.0363
tag36h11:4    | 12.03,1.25,0.50   | 12.00,1.25,0.50   | 0.0288
tag36h11:5    | 15.01,-1.25,0.49   | 15.00,-1.25,0.50   | 0.0139
tag36h11:6    | 15.01,1.23,0.49    | 15.00,1.25,0.50    | 0.0245
-----
Average Tag Position Error: 0.0248 m
=====
```

Noisy dataset — straight trajectory

Rubber duck check — S curve with 5 tags

Pato de borracha: explico que só há ruído extra, a geometria da curva precisa continuar igual nos dois gráficos.

```
=====
FINAL RESULTS
=====
Trajectory RMSE: 0.0000 m
Saving plot to 'slam_final_graph.png'...
Plot saved.

Saving 'dataset_otimizado.yaml' with dataset structure...
File 'dataset_otimizado.yaml' saved successfully (Robot-Ready)!

=====
TAG ESTIMATION ERROR ANALYSIS
=====
Tag Name      | Est (x,y,z)      | True (x,y,z)      | Error (m)
-----
tag36h11:0    | 4.00,1.50,0.50   | 4.00,1.50,0.50   | 0.0017
tag36h11:1    | 6.00,1.00,0.50   | 6.00,1.00,0.50   | 0.0038
tag36h11:2    | 8.01,-1.51,0.50  | 8.00,-1.50,0.50  | 0.0158
tag36h11:3    | 14.03,-0.98,0.50 | 14.00,-1.00,0.50 | 0.0360
tag36h11:4    | 20.02,-0.02,0.50 | 20.00,0.00,0.50  | 0.0250
=====
```

Clean dataset — S curve, 5 tags

```
=====
FINAL RESULTS
=====
Trajectory RMSE: 0.0007 m
Saving plot to 'slam_final_graph.png'...
Plot saved.

Saving 'dataset_otimizado.yaml' with dataset structure...
File 'dataset_otimizado.yaml' saved successfully (Robot-Ready)!

=====
TAG ESTIMATION ERROR ANALYSIS
=====
Tag Name      | Est (x,y,z)      | True (x,y,z)      | Error (m)
-----
tag36h11:0    | 4.00,1.49,0.51   | 4.00,1.50,0.50   | 0.0089
tag36h11:1    | 6.01,0.99,0.50   | 6.00,1.00,0.50   | 0.0173
tag36h11:2    | 8.01,-1.50,0.49  | 8.00,-1.50,0.50  | 0.0108
tag36h11:3    | 14.00,-0.93,0.49 | 14.00,-1.00,0.50 | 0.0748
tag36h11:4    | 20.00,-0.01,0.49 | 20.00,0.00,0.50  | 0.0077
=====
Average Tag Position Error: 0.0239 m
=====
```

Noisy dataset — S curve, 5 tags

Rubber duck check — S curve with 22 tags

Pato de borracha: mais tags para o pato conferir que densidade de medições muda, mas o caminho base permanece.

```
=====
                        FINAL RESULTS
=====
Trajectory RMSE: 0.0002 m
Saving plot to 'slam_final_graph.png'...
Plot saved.

Saving 'dataset_otimizado.yaml' with dataset structure...
File 'dataset_otimizado.yaml' saved successfully (Robot-Ready)!
=====

TAG ESTIMATION ERROR ANALYSIS
=====
Tag Name      | Est (x,y,z)      | True (x,y,z)      | Error (m)
-----
tag36h11:2    | 2.00,1.00,0.50   | 2.00,1.00,0.50   | 0.0017
tag36h11:4    | 4.00,1.00,0.50   | 4.00,1.00,0.50   | 0.0010
tag36h11:6    | 6.00,1.00,0.50   | 6.00,1.00,0.50   | 0.0017
tag36h11:7    | 7.00,-1.00,0.50  | 7.00,-1.00,0.50  | 0.0016
tag36h11:9    | 9.00,-1.00,0.50  | 9.00,-1.00,0.50  | 0.0011
tag36h11:11   | 11.00,-1.00,0.50 | 11.00,-1.00,0.50 | 0.0010
tag36h11:13   | 13.00,-1.00,0.50 | 13.00,-1.00,0.50 | 0.0028
tag36h11:14   | 14.00,1.00,0.50  | 14.00,1.00,0.50  | 0.0013
tag36h11:16   | 16.00,1.00,0.50  | 16.00,1.00,0.50  | 0.0010
tag36h11:18   | 18.00,1.00,0.50  | 18.00,1.00,0.50  | 0.0013
tag36h11:19   | 19.00,-1.00,0.50 | 19.00,-1.00,0.50 | 0.0020
tag36h11:20   | 20.01,1.00,0.50  | 20.00,1.00,0.50  | 0.0068
tag36h11:21   | 21.00,-1.00,0.50 | 21.00,-1.00,0.50 | 0.0012
=====
Average Tag Position Error: 0.0019 m
=====
```

Clean dataset — S curve, 22 tags

```
=====
                        FINAL RESULTS
=====
Trajectory RMSE: 0.0037 m
Saving plot to 'slam_final_graph.png'...
Plot saved.

Saving 'dataset_otimizado.yaml' with dataset structure...
File 'dataset_otimizado.yaml' saved successfully (Robot-Ready)!
=====

TAG ESTIMATION ERROR ANALYSIS
=====
Tag Name      | Est (x,y,z)      | True (x,y,z)      | Error (m)
-----
tag36h11:2    | 2.00,1.01,0.50   | 2.00,1.00,0.50   | 0.0098
tag36h11:4    | 4.01,1.00,0.50   | 4.00,1.00,0.50   | 0.0065
tag36h11:6    | 6.00,1.00,0.50   | 6.00,1.00,0.50   | 0.0037
tag36h11:7    | 7.00,-1.00,0.50  | 7.00,-1.00,0.50  | 0.0033
tag36h11:9    | 9.00,-1.00,0.50  | 9.00,-1.00,0.50  | 0.0056
tag36h11:11   | 11.00,-0.99,0.50 | 11.00,-1.00,0.50 | 0.0087
tag36h11:13   | 13.01,-1.01,0.50 | 13.00,-1.00,0.50 | 0.0092
tag36h11:14   | 14.01,0.99,0.49  | 14.00,1.00,0.50  | 0.0133
tag36h11:16   | 15.98,1.00,0.49  | 16.00,1.00,0.50  | 0.0192
tag36h11:18   | 18.00,1.00,0.49  | 18.00,1.00,0.50  | 0.0074
tag36h11:19   | 19.01,-1.02,0.49 | 19.00,-1.00,0.50 | 0.0273
tag36h11:20   | 20.00,0.98,0.54  | 20.00,1.00,0.50  | 0.0433
tag36h11:21   | 21.01,-1.00,0.52 | 21.00,-1.00,0.50 | 0.0175
=====
Average Tag Position Error: 0.0134 m
=====
```

Noisy dataset — S curve, 22 tags

Phase 5: Path recovery algorithm

Goal: Recover when the robot leaves the taught path

- ▶ **Recovery logic:** Simple rules to steer back to the recorded path.

Phase 5: Path recovery algorithm

Goal: Recover when the robot leaves the taught path

- ▶ **Recovery logic:** Simple rules to steer back to the recorded path.
- ▶ **Closest waypoint search:** Use a quick KD-Tree to find the nearest waypoint.

Phase 5: Path recovery algorithm

Goal: Recover when the robot leaves the taught path

- ▶ **Recovery logic:** Simple rules to steer back to the recorded path.
- ▶ **Closest waypoint search:** Use a quick KD-Tree to find the nearest waypoint.

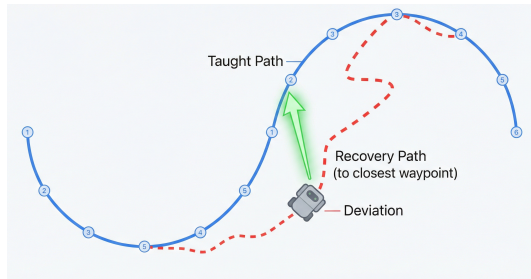


Fig. 10 — Returning to the original path after deviation

Phase 5: Path recovery algorithm

Goal: Recover when the robot leaves the taught path

- ▶ **Recovery logic:** Simple rules to steer back to the recorded path.
- ▶ **Closest waypoint search:** Use a quick KD-Tree to find the nearest waypoint.
- ▶ **Safety validation:** Test recovery moves in sim to avoid bad turns.

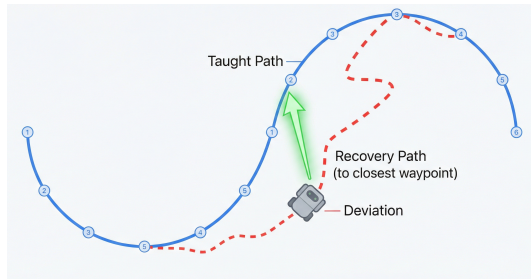


Fig. 10 — Returning to the original path after deviation

Phase 6: System integration

Goal: Demonstrate full teach and repeat autonomy

- ▶ **Controller integration:** Mix Pure Pursuit with the perception and pose stack.

Phase 6: System integration

Goal: Demonstrate full teach and repeat autonomy

- ▶ **Controller integration:** Mix Pure Pursuit with the perception and pose stack.
- ▶ **Autonomous replay:** Let the robot drive the recorded path on its own.

Phase 6: System integration

Goal: Demonstrate full teach and repeat autonomy

- ▶ **Controller integration:** Mix Pure Pursuit with the perception and pose stack.
- ▶ **Autonomous replay:** Let the robot drive the recorded path on its own.

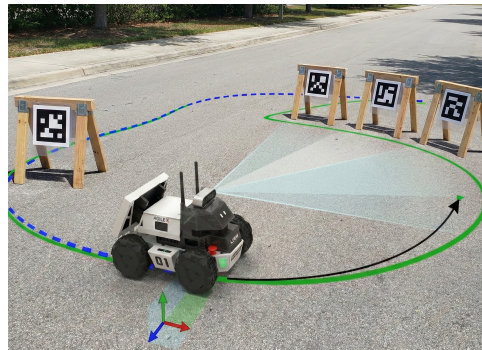


Fig. 11 — Fully integrated teach and repeat behavior

Phase 6: System integration

Goal: Demonstrate full teach and repeat autonomy

- ▶ **Controller integration:** Mix Pure Pursuit with the perception and pose stack.
- ▶ **Autonomous replay:** Let the robot drive the recorded path on its own.
- ▶ **Final system:** A teach & repeat pipeline anyone can run without coding.

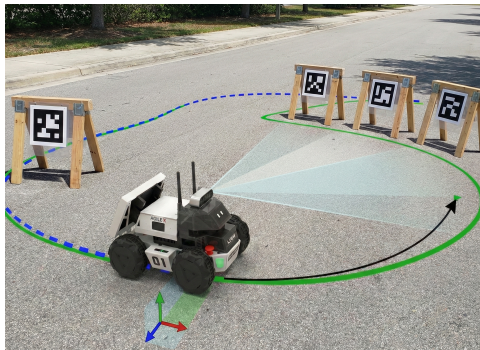


Fig. 11 — Fully integrated teach and repeat behavior

- [1] R. Adámek, M. Brabc, P. Vávra, B. Dobossy, M. Formánek, and F. Radil, “Analytical Models for Pose Estimate Variance of Planar Fiducial Markers for Mobile Robot Localisation”, *Sensors*, vol. 23, no. 12, p. 5746, 2023. DOI: 10.3390/s23125746
- [2] B. S. Herrera, M. García, W. Chamorro, and D. Maldonado, “Visual Mapping and Autonomous Navigation Using AprilTags in Omnidirectional Mobile Robots: A Realistic ROS-Gazebo Simulation Framework”, *Engineering Proceedings*, vol. 115, no. 1, p. 5, 2025. DOI: 10.3390/engproc2025115005
- [3] S. Thrun, W. Burgard, and D. Fox, *Probabilistic Robotics*, MIT Press, 2005. DOI: 10.5555/1625995.1626105
- [4] B. Pfrommer and K. Daniilidis, “TagSLAM: Robust SLAM with Fiducial Markers”, *arXiv preprint arXiv:1910.00679*, 2019. DOI: 10.48550/arXiv.1910.00679

Thank you for listening!

Any questions?

Joao Pedro Martins do Lago Reis | Yann Kelvem da Silva Ramos
AprilTag-based Trajectory Tracking for the LIMOS Robot